

异步 Python 服务实战

构建具有并发边界、超时、取消、重试和可观测性的异步服务。

- 明确任务生命周期
- 限制并发
- 传播取消
- 观察完整请求

Event Loop 与协程

Event Loop 调度可暂停的协程，让等待 I/O 的时间服务其他任务。

- coroutine
- task
- await
- cooperative scheduling

async/await 执行模型

await 只在可等待边界让出控制权，CPU 密集任务仍需独立执行策略。

```
async def load_all(ids):  
    async with asyncio.TaskGroup() as group:  
        tasks = [group.create_task(load(i)) for i in ids]  
    return [task.result() for task in tasks]
```

并发任务与背压

无限创建 Task 会耗尽连接和内存，需要 semaphore 或队列限制并发。

- 设置并发上限
- 限制队列长度
- 拒绝或降级过载
- 记录等待时间

超时边界

为外部操作和整体请求分别设置超时，避免单个依赖拖垮服务。

```
async with asyncio.timeout(2.0):  
    result = await client.fetch()
```

取消与资源清理

取消是正常控制流，资源必须在 finally 或异步上下文中可靠释放。

- 传播 CanceledError
- 关闭连接
- 回滚临时状态
- 不要吞掉取消

异常传播

并发任务的失败需要聚合、分类并映射到稳定的服务错误。

- 保留根因
- 区分局部与整体失败
- 记录 task identity
- 返回可恢复信息

有界重试与 Backoff

只对瞬时故障重试，并使用指数退避、抖动和最大尝试次数。

```
for attempt in range(max_attempts):
    try:
        return await operation()
    except TransientError:
        await asyncio.sleep(backoff(attempt))
```


日志、指标与 Trace

统一 correlation ID 连接一次请求的日志、指标和跨步骤 Trace。

- 结构化日志
- 延迟和错误率指标
- span 记录外部调用
- 避免敏感数据

可靠异步服务练习

实现有界并发 HTTP 聚合服务，并验证超时、取消、重试与观测行为。

- TaskGroup 管理任务
- Semaphore 限制并发
- 超时触发取消
- 测试失败与恢复